# Python OOP 입문

## 클래스 · 상속 · 추상화

cchyun@gmail.com

# Part 1. OOP 필요성과 클래스 기초

# OOP가 왜 필요한가? — 절차형 코드와의 비교

- **로봇 프로그래밍 = Sense(인지) → Think(판단) → Act(동작) 의 무한 반복**

  - **절차형 프로그래밍(Procedural)**
    - 명령어를 순서대로 실행 → 초기 학습·간단한 스크립트에 적합
    - 시스템이 복잡해질수록 **상태 관리와 코드 재사용이 급격히 어려워짐**
    - 예: 센서 3개 → `sensor1_dist`, `sensor2_dist`, `sensor3_dist` … 10개가 되면?

  - **객체지향 프로그래밍(OOP)**
    - 로봇 부품(센서, 모터 등)을 **객체(Object)** 로 묶어 관리
    - **데이터(상태)와 기능(동작)을 한곳에 모아** 복잡한 시스템을 체계적으로 설계

# 로봇 개발에서 OOP가 특히 중요한 이유

- **로봇 = 하드웨어와 상호작용하는 복합 시스템 → OOP가 필수**
  - **컴포넌트 모델링**
    - 모터·카메라·IMU 등 각 부품을 클래스(설계도)로 모델링 → 인스턴스로 관리
  - **캡슐화(Encapsulation)**
    - 복잡한 하드웨어 통신 로직은 클래스 내부에 숨기고
    - 외부에는 `.get_distance()` 같은 **단순한 API만 노출**
  - **하드웨어 추상화 계층(HAL)**
    - 모터 드라이버가 바뀌어도 `.set_velocity` 인터페이스만 유지하면 **전체 코드 수정 불필요**
  - **다형성(Polymorphism)**
    - LiDAR·초음파·범퍼를 **동일한 방식으로 제어 루프에서 호출** 가능

# 클래스(Class)와 인스턴스(Instance)의 차이

- **비유: "붕어빵 틀" vs "붕어빵"**

  - **클래스(Class) — 설계도 / 틀**
    - "모든 거리 센서는 이름이 있고, 값을 읽는 기능이 있다" 는 **정의 자체**
    - 메모리에 실체가 없는 상태

  - **인스턴스(Instance) — 실체**
    - 설계도를 바탕으로 메모리에 **실제로 생성된 객체**
    - 예: "로봇 전면에 부착된 1번 초음파 센서"
    - 하나의 클래스로 **여러 개의 인스턴스** 생성 가능

```
front_sensor = DistanceSensor("Front_Lidar")    # 인스턴스 1
back_sensor  = DistanceSensor("Back_Ultrasonic") # 인스턴스 2
```

# `__init__` 과 `self` 의 역할

- **Python 클래스 구조의 두 가지 핵심**

  - `__init__` **(생성자)**
    - 객체가 **처음 만들어질 때 자동으로 실행**되는 함수
    - 로봇 개발에서 주로: 센서 포트 설정 / 통신 초기화 / 캘리브레이션 값 설정

  - `self` **— "나 자신"을 가리키는 포인터**
    - 클래스 내부 변수·함수가 **어느 인스턴스의 것인지** 구별
    - 예: "내( `self` ) 포트 번호에서 값을 가져와"
    - Python 관례상 **첫 번째 인자로 항상** `self` **사용**

```python
def __init__(self, name, min_range=0.0, max_range=200.0):
    self.name      = name          # 인스턴스에 name 속성 부여
    self.min_range = min_range
    self.last_value = 0.0
```

# 클래스 변수 vs 인스턴스 변수

- **로봇 데이터 관리 시 명확히 구분 필요**

  - **인스턴스 변수** — 각 인스턴스마다 독립적인 값
    - `self.distance` : 왼쪽·오른쪽 센서의 측정값은 서로 다름
    - `__init__` 안에서 `self.변수명 = 값` 으로 정의

  - **클래스 변수** — 해당 클래스의 모든 인스턴스가 **공유**
    - `UNIT = "cm"` : 시스템 공통 단위, 전체 센서 개수 등
    - 클래스 본체(메서드 밖)에서 정의

  - ⚠️ **주의사항**
    - **가변 객체(list, dict 등)를 클래스 변수로 사용하면**
    - 모든 인스턴스가 공유 → **의도치 않은 데이터 오염** 발생 가능

# 입문자가 가장 많이 혼란스러워하는 포인트

- **자주 발생하는 3가지 혼동**
  - ① `self` **인자 생략 혼동**
    - 함수 정의 시엔 `self`를 쓰고, 호출 시엔 `self`를 쓰지 않음
    - Python이 호출 시 **자동으로 첫 번째 인자에 객체 자신을 전달**

```
front_sensor.get_reading()  # self 없이 호출 → Python이 자동 처리
```

  - ② **변수 범위(Scope) 혼동**
    - 클래스 안에서 `self.`을 붙이지 않으면 **일반 지역 변수로 취급**
    - 함수가 끝나는 순간 사라짐 → 인스턴스에 저장되지 않음
  - ③ **캡슐화의 이점 이해 부족**
    - 변수와 함수를 왜 하나로 묶어야 하는지 체감하는 데 시간이 걸림

# (실습) 로봇 거리 센서 클래스 예제 (1/2)

```python
import random  # 센서 값 시뮬레이션

class DistanceSensor:
    UNIT = "cm"  # 클래스 변수: 모든 센서가 공유

    def __init__(self, name, min_range=0.0, max_range=200.0):
        self.name       = name        # 인스턴스 변수: 센서 이름
        self.min_range  = min_range   # 인스턴스 변수: 최소 범위
        self.max_range  = max_range   # 인스턴스 변수: 최대 범위
        self.last_value = 0.0         # 인스턴스 변수: 최근 측정값

    def get_reading(self):
        # 실제 환경에서는 하드웨어 통신 코드가 들어감
        raw_value = random.uniform(self.min_range, self.max_range)
        self.last_value = round(raw_value, 2)
        print(f"[{self.name}] 측정 완료: {self.last_value} {self.UNIT}")
        return self.last_value
```

# (실습) 로봇 거리 센서 클래스 예제 (2/2)

```
# 인스턴스 생성 (두 개의 센서 하드웨어 모델링)
front_sensor = DistanceSensor("Front_Lidar",      10.0, 500.0)
back_sensor  = DistanceSensor("Back_Ultrasonic",   2.0, 150.0)

# 각 센서 인스턴스의 동작 호출 (self는 자동 전달)
val1 = front_sensor.get_reading()
val2 = back_sensor.get_reading()

# 인스턴스 변수 확인
print(f"전방 센서 최근 값: {front_sensor.last_value}")
print(f"후방 센서 최근 값: {back_sensor.last_value}")
```

- **핵심 포인트**
  - 각 센서는 자신만의 `name`·범위를 가짐 → **인스턴스 변수**
  - `get_reading()`이라는 **공통 기능(메서드)** 은 클래스에서 한 번만 정의
  - 이 구조가 수백 개의 센서를 다룰 때 **시스템 안정성의 기초**

# Part 2. 상속과 추상화

# 상속이 왜 필요한가? — 로봇 센서 클래스 계층 구조

- **로봇에는 LiDAR·카메라·IMU 등 다양한 센서가 장착됨**
  - 각기 다르지만 '데이터 읽기', '상태 확인' 같은 **공통된 행위를 공유**

  - **코드 재사용**
    - 공통 기능을 `BaseSensor`에 정의 → 자식 클래스에서 상속
    - **중복 코드 제거** + 한 곳만 수정하면 전체 반영

  - **계층적 구조**
    - 일반 하드웨어 범주(부모) → 특수 하드웨어 모델(자식)
    - 시스템을 **직관적으로 이해** 가능

  - **유지보수 용이성**
    - 공통 로직 수정 시 **부모 클래스만 수정** → 자식 클래스 전체에 자동 반영

# `super()` 의 역할과 사용 방법

- `super()` — **자식 클래스에서 부모 클래스에 접근하는 함수**
  - **역할**
    - `__init__` 내에서 **부모 클래스의 초기화 로직을 실행**
    - 호출하지 않으면 부모에서 정의된 상태값이 자식 인스턴스에 올바르게 설정되지 않음
  - **사용 방법**

```
class LidarSensor(BaseSensor):
    def __init__(self, name, scan_range):
        super().__init__(name)    # 부모 생성자 먼저 실행
        self.scan_range = scan_range  # 자식만의 속성 추가
```

  - ⚠️ **가장 흔한 실수: `super().__init__()` 호출 누락**
    - 부모 클래스의 설정값이 초기화되지 않아 런타임 에러 발생

# 메서드 오버라이딩(Method Overriding) — 개념과 활용

- **부모 클래스의 메서드를 자식 클래스에서 재정의**

  - **활용 예시**
    - 모든 센서가 `.get_data()` 메서드를 가지지만
    - LiDAR → 거리 배열 반환 / Camera → 이미지 배열 반환
    - **자식 클래스에서 오버라이딩** → 하드웨어 특화 로직 구현

  - **다형성(Polymorphism)**
    - 제어 루프에서 **센서 종류에 상관없이 동일한 이름의 메서드 호출**
    - 각 객체가 **자신의 오버라이딩된 로직을 실행**

```
for sensor in sensors:
    print(sensor.get_data())  # 각자 다른 동작 수행
```

# 추상 클래스(ABC)가 왜 필요한가? — 로봇 컴포넌트 설계

- **추상 클래스 = 직접 객체를 생성할 수 없는 "설계 도면"**

  - **인터페이스 강제**
    - 자식 클래스가 반드시 구현해야 할 메서드를 **강제 규정**
    - 로봇 소프트웨어가 `get_data` 메서드의 존재를 믿고 동작 가능

  - **안전성**
    - 새 센서 드라이버 개발 시 필수 메서드를 구현하지 않으면 **즉시 에러 발생**
    - 설계 의도대로 개발하도록 유도

  - **구현 방법 (`abc` 모듈)**

```python
from abc import ABC, abstractmethod

class BaseSensor(ABC):
    @abstractmethod
    def get_data(self):
        """자식 클래스에서 반드시 구현해야 함"""
        pass
```

# 다중 상속의 개념과 주의사항 (MRO)

- **Python은 두 개 이상의 부모 클래스로부터 상속받는 다중 상속을 지원**
  - **개념**

```
class WirelessSensor(BluetoothMixin, BaseSensor):
    pass
```

  - ⚠️ **주의사항 — MRO(Method Resolution Order)**
    - 여러 부모가 동일한 이름의 메서드를 가질 경우 혼란 발생 가능
    - Python은 **MRO 규칙**으로 어떤 부모의 메서드를 먼저 실행할지 결정
    - `ClassName.__mro__` 로 확인 가능
  - **실무 팁**
    - 다중 상속보다는 **합성(Composition)** 이 적절한 경우가 더 많음
    - 단순 기능 재사용 목적이라면 상속보다 **객체를 속성으로 포함**하는 방식 고려

# 입문자가 상속에서 자주 하는 실수와 해결법

- **상속 학습 시 자주 발생하는 3가지 실수**

  - **① 부모 생성자 호출 누락**
    - `super().__init__()` 을 빠뜨리면 부모 클래스의 설정값이 초기화되지 않음
    - **해결:** 자식 `__init__` 첫 줄에 항상 `super().__init__()` 호출

  - **② 잘못된 클래스 명명 규칙**
    - 클래스 이름은 반드시 `UpperCamelCase` 로 작성해야 가독성이 높아짐
    - ✅ `LidarSensor` ❌ `lidar_sensor`, `lidarsensor`

  - **③ 불필요한 상속 남용**
    - 단순 기능 재사용을 위한 상속 남발은 복잡도만 증가
    - **"is-a 관계"** (LiDAR is-a Sensor)일 때만 상속 사용
    - **"has-a 관계"** (Robot has-a Motor)라면 합성(Composition) 사용

# (실습) 로봇 센서 상속 구조 설계 예제 (1/2)

```python
from abc import ABC, abstractmethod

# 1단계: 추상 베이스 클래스 정의 (설계도)
class BaseSensor(ABC):
    def __init__(self, name):
        self.name = name
        print(f"[{self.name}] 센서가 초기화되었습니다.")

    @abstractmethod
    def get_data(self):
        """자식 클래스에서 반드시 구현해야 함"""
        pass

# 2단계: LiDAR 클래스 (상속 및 오버라이딩)
class LidarSensor(BaseSensor):
    def __init__(self, name, scan_range):
        super().__init__(name)          # 부모 생성자 실행
        self.scan_range = scan_range

    def get_data(self):
        return f"{self.name}: {self.scan_range}도 스캔 완료"
```

# (실습) 로봇 센서 상속 구조 설계 예제 (2/2)

```python
# 3단계: Camera / IMU 클래스
class CameraSensor(BaseSensor):
    def get_data(self):
        return f"{self.name}: RGB 이미지 프레임 획득"

class IMUSensor(BaseSensor):
    def get_data(self):
        return f"{self.name}: 가속도 및 자세 데이터 측정"

# 4단계: 다형성 활용 — 제어 루프
sensors = [
    LidarSensor("Hokuyo_LiDAR", 360),
    CameraSensor("Realsense_RGB"),
    IMUSensor("Bosch_IMU"),
]

for sensor in sensors:
    print(sensor.get_data())  # 각 클래스의 오버라이딩된 메서드 실행
```

- **이 구조 = 하드웨어 추상화 계층(HAL)**
  - 제어 시스템이 센서의 구체적인 종류를 몰라도 `get_data()` 호출로 정보 획득

# 실습 체크리스트

- **클래스 설계 및 구현 완료 여부 확인**
  - **① 클래스 정의**
    - 로봇 부품을 추상화하여 `UpperCamelCase` 형식으로 클래스 이름을 지었는가?
  - **② 생성자 활용**
    - `__init__` 메서드로 센서의 초기값(포트 번호, 해상도 등)을 설정했는가?
  - **③ 상속 구현**
    - 공통 기능을 가진 `Base` 클래스를 만들고, 이를 상속받아 구체적인 센서를 구현했는가?
  - **④ 메서드 구현**
    - `self`를 첫 번째 인자로 사용하여 인스턴스의 데이터에 접근하는 메서드를 정의했는가?
  - **⑤ 객체 생성 및 테스트**
    - 정의한 클래스로 실제 인스턴스를 생성하고 메서드를 호출하여 결과를 확인했는가?